Hugging Face Python Scraper

Enhancement Implementation Summary

Project documentation | 12 November 2026

This document summarises the eight enhancements discussed and implemented. Each section explains the enhancement and then provides the relevant code additions or implementation pattern.

Implementation overview

No.	Enhancement	Status
1	Incremental scraping and duplicate prevention	Implemented; existing records are loaded, known Case Links are skipped, and new records are merged at export.
2	HTTP request timeout handling	Implemented in page-loading and PDF download requests.
3	Improved retry mechanism	Implemented in the active page-loading method with bounded retries and backoff.
4	Extracted data validation	Implemented before valid records are appended.
5	Failed URL tracking and reporting	Implemented for listing-page and paper-page failures.
6	User-defined date range	Implemented and exercised with a selected date range.
7	User-defined PDF download location	Implemented; the selected folder is passed to the download method.
8	PDF URL accessibility verification and failure reporting	Implemented; HTTP response and PDF indicators are checked, with failures reported.

1. Incremental Scraping and Duplicate Prevention

The scraper loads the existing CSV, builds a set of processed Case Link values, skips papers already present, then merges new records with the old dataset and removes duplicate Case Links.

Load existing data (Main.py):

csv_file = "Huggingface_Data - MT-1354.csv" try: existing_df = pd.read_csv(csv_file) existing_case_link = set(existing_df["Case Link"].dropna().astype(str)) print(f"Previously scraped papers: {len(existing_case_link)}") except FileNotFoundError: existing_df = pd.DataFrame() existing_case_link = set() df_data_set = []

Skip a previously scraped paper:

if pappers in existing_case_link: print(f"Skipping already scraped paper: {pappers}") continue

Add a successfully processed link:

existing_case_link.add(pappers)

Merge and export:

new_df = pd.DataFrame(df_data_set, columns=["Title", "published", "Submitted On", "Abstract", "Page URL", "Case Link", "pdf link", "Date"]) if not existing_df.empty: final_df = pd.concat([existing_df, new_df], ignore_index=True) else: final_df = new_df final_df.drop_duplicates(subset=["Case Link"], keep="first", inplace=True) final_df.to_csv(csv_file, index=False)

2. HTTP Request Timeout Handling

Connection and read timeouts prevent HTTP requests from waiting indefinitely. The tuple (5, 20) sets a 5-second connection timeout and a 20-second read timeout.

In the active LoadPage() method (Processor.py):

response = requests.get(url, timeout=(5, 20))

In download_pdf() (Processor.py):

response = requests.get(url, timeout=(5, 20))

3. Improved Retry Mechanism

The active page-loading method uses up to three attempts and exponential backoff for temporary failures. It handles HTTP 429 and 5xx responses and respects an integer Retry-After header when supplied. The snippet below is the core retry pattern; retain the additional URL validation, logging, and failure-detail handling in the actual method.

Core retry pattern (Processor.py):

MAX_RETRIES = 3 for attempt in range(1, MAX_RETRIES + 1): try: response = requests.get(url, timeout=(5, 20)) if response.status_code == 429 or response.status_code >= 500: retry_after = response.headers.get("Retry-After") if retry_after and retry_after.isdigit(): wait_time = int(retry_after) else: wait_time = 2 ** attempt time.sleep(wait_time) continue if not self.ValidateResponse(url, response): return None return response except (requests.exceptions.Timeout, requests.exceptions.ConnectionError): time.sleep(2 ** attempt) continue

4. Extracted Data Validation

A validation method checks required paper fields, URL validity, and date formatting before a record is accepted. Main.py calls the method and skips records that fail validation.

Validation method (Processor.py):

def ValidateExtractedData(self, title, published_on, submitted_on, abstract, paper_url, pdf_url): errors = [] if not title or not title.strip(): errors.append("Missing title") if not published_on: errors.append("Missing publication date") if not abstract or not abstract.strip(): errors.append("Missing abstract") if not paper_url or not checkers.is_url(paper_url): errors.append("Invalid paper URL") if pdf_url and not checkers.is_url(pdf_url): errors.append("Invalid PDF URL") try: datetime.strptime(published_on, "%Y-%m-%d") except (ValueError, TypeError): errors.append("Invalid publication date format") if submitted_on: try: datetime.strptime(submitted_on, "%Y-%m-%d") except (ValueError, TypeError): errors.append("Invalid submitted date format") return len(errors) == 0, errors

Call it from Main.py before appending:

is_valid, validation_errors = (HuggingFace_Scrape.ValidateExtractedData(worker, title, published, submitted_on, abstract, pappers, pdf_link)) if not is_valid: worker.logger.warning(f"Invalid extracted data: {validation_errors}") continue

5. Failed URL Tracking and Reporting

Failed listing-page and paper-page requests are recorded with URL, URL type, failure reason, status code, attempt count, and status. The details are exported to failed_urls.csv.

Initialize the list (Main.py):

failed_url_details = []

When a listing page fails:

failure_info = worker.last_failure_info.copy() failure_info["URL Type"] = "Listing Page" failure_info["Status"] = "Failed" failed_url_details.append(failure_info)

When a paper page fails:

failure_info = worker.last_failure_info.copy() failure_info["URL Type"] = "Paper Page" failure_info["Status"] = "Failed" failed_url_details.append(failure_info)

Export the report:

failed_df = pd.DataFrame(failed_url_details, columns=["URL", "URL Type", "Failure Reason", "Status Code", "Attempts", "Status"]) failed_df.to_csv("failed_urls.csv", index=False)

6. User-Defined Date Range Input

The hardcoded date range was replaced with start-date and end-date prompts. The method validates the date order and generates every date in the range, inclusive.

Collect dates in Main.py:

start_date = input("Enter start date (YYYY-MM-DD): ").strip() end_date = input("Enter end date (YYYY-MM-DD): ").strip() date_range = HuggingFace_Scrape.get_date_range(worker, start_date, end_date)

Generate dates in Processor.py:

def get_date_range(self, start_date, end_date): try: date_ranges = [] start_date = datetime.strptime(start_date, "%Y-%m-%d") end_date = datetime.strptime(end_date, "%Y-%m-%d") if start_date > end_date: raise ValueError("Start date cannot be after end date") current_date = start_date while current_date <= end_date: date_ranges.append(current_date.strftime("%Y-%m-%d")) current_date += timedelta(days=1) return date_ranges except Exception as ex: self.logger.critical(f"Invalid date range: {ex}") raise

7. User-Defined PDF Download Location

The user enters a destination folder instead of relying on a hardcoded path. The download method creates the folder if needed, builds the output filename, and skips files that already exist.

Collect the folder path in Main.py:

pdf_download_path = input("Enter PDF download folder path: ").strip().strip("") if not pdf_download_path: print("PDF download location cannot be empty.") exit()

Prepare the destination in download_pdf() (Processor.py):

if not path: self.logger.error("PDF download path is empty.") return False os.makedirs(path, exist_ok=True) location = os.path.join(path, file_name) if os.path.isfile(location): print(f"PDF already exists. Skipping: {file_name}") return True

Pass the selected path from Main.py:

download_success = HuggingFace_Scrape.download_pdf(worker, pdf_link, file_name + ".pdf", pdf_download_path)

8. PDF URL Accessibility Verification and Failure Reporting

Before downloading, the scraper checks the HTTP response and whether the returned content appears to be a PDF. It checks the Content-Type header and the %PDF- file signature. If verification fails, the reason is retained for the failure report.

Verification method (Processor.py):

def verify_pdf_url(self, url): try: response = requests.get(url, stream=True, timeout=(5, 20)) if response.status_code == 200: content_type = response.headers.get("Content-Type", "").lower() first_bytes = next(response.iter_content(5), b"") if ("pdf" in content_type or first_bytes.startswith(b"%PDF-")): response.close() return True, "PDF accessible" response.close() return False, f"Invalid content type: {content_type}" response.close() return False, f"HTTP status: {response.status_code}" except requests.exceptions.Timeout: return False, "Request timeout" except requests.exceptions.RequestException as ex: return False, str(ex)

Verify before downloading (inside download_pdf()):

is_accessible, reason = self.verify_pdf_url(url) if not is_accessible: self.logger.warning(f"PDF URL verification failed: {url} - {reason}") print(f"PDF unavailable: {url} - {reason}") self.last_pdf_failure = { "URL": url, "File Name": file_name, "Failure Reason": reason } return False

Collect failures in Main.py:

failed_pdf_details = [] # Before each download: worker.last_pdf_failure = None # After the download attempt: if not download_success: failure_info = worker.last_pdf_failure if failure_info: failure_info["Date"] = date failed_pdf_details.append(failure_info.copy())

Export the PDF failure report:

failed_pdf_df = pd.DataFrame(failed_pdf_details, columns=["Date", "URL", "File Name", "Failure Reason"]) failed_pdf_df.to_csv("failed_pdf_report.csv", index=False)

Observed execution results

In the observed run, the terminal showed successful process completion, 2,113 total records saved, and generation of the PDF failure report. Existing PDFs were skipped. One PDF failure was recorded, and the user checked that the corresponding PDF was unavailable on the website.

Scope and testing notes

- The retry mechanism retries requests during the current run. Automatic reprocessing of URLs from a previous failed_urls.csv report has not been implemented.
- The main CSV is merged and saved at the end of the run. Periodic crash-safe checkpoint saving has not been implemented.
- The current PDF design uses one user-selected destination folder, not date-wise subfolders.
- Code snippets are implementation excerpts, not full replacements for Main.py or Processor.py. Preserve surrounding existing logic and exception handling.
- A successful syntax check does not by itself prove that every error-handling path has been tested.

Output files

- Huggingface_Data - MT-1354.csv — main scraped dataset.

- failed_urls.csv — listing-page and paper-page failure report.
- failed_pdf_report.csv — PDF verification/download failure report.
- User-selected PDF folder — downloaded PDF files.